

06_autocommit_daemon_and_branch_safety

06 — “Auto-commit” mechanism forensics + feature-branch safety (\$11.4.181 prep)

Revision: 1 **Captured:** 2026-07-06 (UTC) on host the-factory = the machine running this repo **Repo:** /home/milos/Factory/projects/tools_and_research/helix_code **Method:** \$11.4.6 — every claim below is backed by pasted evidence or explicitly marked UNCONFIRMED: . Evidence copies live under ./evidence/daemon/.

0. TL;DR

- The commits titled **Auto-commit** are produced by a reusable **commit shell-command toolkit** installed on \$PATH from project_toolkit, **not** by any OS-level daemon on this host.
 - **Branch targeting:** it commits to the **CURRENT branch (HEAD)** — plain git commit -m. There is **NO** git checkout, git switch, git reset, or branch hard-targeting anywhere in the chain. → **A feature branch is safe: Auto-commit lands on whatever branch is checked out.**
 - **Add scope:** **git add .** (everything in the current working tree, tracked + untracked, incl. submodule GITLINK pointer bumps). This is broad, but branch-safe.
 - **Push:** after a successful commit it runs push_all.sh → git push <remote> **of the current branch** to every configured upstream + git push --tags. **No --force** (\$11.4.113-compatible).
 - **Cadence / “daemon”:** UNCONFIRMED: there is **no cron, no systemd timer (user or system), no at job, no file-watcher, no shell loop** invoking it. Cadence is irregular (3 commits inside 4 min on 2026-07-05, none on 2026-07-06, prior gap back to 2026-07-02). The trigger is an **external orchestrator** — most plausibly the running Claude Code agent sessions (helix_code, helix_terminator, atmosphere_t1) calling commit/cmt as part of their work loop, and/or a cross-host rsync sync (the Test User <test@example.com> “sync:” commits). See \$5.
-

1. The invocation chain (file:line evidence)

Four scripts form the chain. All emit the literal `git log` subject **Auto-commit** (no period) when invoked with no argument.

1a. `cmt` — repo-root convenience wrapper (NOT the auto path)

`/home/milos/Factory/projects/tools_and_research/helix_code/cmt`

```
15     MESSAGE="Auto-commit."           # NOTE: has a trailing
    period
34  echo "Pushing: '$MESSAGE'" && \
35  commit "$MESSAGE" && \             # delegates to the
    `commit` command on PATH
```

`cmt` always passes a message ("Auto-commit." with a period, or `$DEFAULT_COMMIT_MESSAGE/$1`). The real Auto-commit commits have **no period**, so `cmt` is a manual helper, not the source of the observed commits.

1b. `commit` — the PATH command → `commit.sh`

which `commit` → `/home/milos/Factory/project_toolkit/Upstreamable/commit`

```
9   SCRIPT_COMMIT="$SUBMODULES_HOME/Upstreamable/commit.sh"
19  bash "$SCRIPT_COMMIT" "$1"         # if arg given
23  bash "$SCRIPT_COMMIT"              # if NO arg ← the auto
    path
```

Installed on `$PATH` via `.bashrc:228 export SUBMODULES_HOME=/home/milos/Factory/project_toolkit` (and `project_toolkit/Upstreamable` is on `$PATH`).

1c. `Upstreamable/commit.sh` — builds the default message

`/home/milos/Factory/project_toolkit/Upstreamable/commit.sh`

```
35  SCRIPT_COMMIT="$SUBMODULES_HOME/Software-Toolkit/Utils/Git/
    commit.sh"
39  MESSAGE="Auto-commit $SESSION"     # $SESSION is EMPTY in
    this env → "Auto-commit "
43  MESSAGE="$1"                      # overridden only if an
    arg was passed
46  if ! bash "$SCRIPT_COMMIT" "$MESSAGE"; then
```

`$SESSION` resolves empty (`bash -lc 'echo [$SESSION]' → []`), so `MESSAGE="Auto-commit "` and git trims the trailing space → **exactly Auto-commit**. This matches the observed subjects byte-for-byte.

1d. Software-Toolkit/Utils/Git/commit.sh — THE committer (add + commit + push)

/home/milos/Factory/project_toolkit/Software-Toolkit/Utils/Git/commit.sh

```
17 SESSION=$((($(date +%s%N)/1000000)) # ms timestamp (used
    only if no arg)
18 MESSAGE="Auto-commit $SESSION"
20 if [ -n "$1" ]; then
22     MESSAGE="$1" # arg "Auto-commit"
    from 1c wins → "Auto-commit"
25 if git add .; then # ← ADD SCOPE:
    everything under CWD
27     if git commit -m "$MESSAGE"; then # ← CURRENT BRANCH; no
        checkout/reset
29         bash "$SCRIPT_PUSH_ALL" # ← push after commit
```

Behavioral facts proven by this file: - **git add .** — stages all changes (modified/new/deleted, tracked + untracked) under the current directory. Run from repo root, this **includes submodule pointer (GITLINK) bumps** and any dirty top-level files. It does **NOT** `git submodule foreach` — it does not commit *inside* submodules, it only records their pointer changes in the meta-repo. - **git commit -m** — commits to **HEAD / current branch**. No `-b`, no checkout, no switch, no reset. Confirmed by full read (only these three git verbs: add, commit, and push in 1e). - No branch name is ever referenced → it **cannot** hard-target main; it commits wherever you are.

1e. Software-Toolkit/Utils/Git/push_all.sh — the push

/home/milos/Factory/project_toolkit/Software-Toolkit/Utils/Git/push_all.sh

```
122 if echo "Upstream '$NAME': $UPSTREAM" && git push
    "$NAME"; then
124     git config pull.rebase false && git fetch && git pull
        && echo "'$NAME': OK"
155 if git push --tags >/dev/null 2>&1; then
```

- `git push "$NAME"` — pushes the **current branch** (no explicit refspec → uses branch's configured upstream/push.default) to each remote named

- by an `upstreams/*.sh` recipe file (loops over `upstreams/` or legacy `Upstreams/`, §11.4.29-aware, lines 19–29 / 130–151).
 - Then `git fetch && git pull` on the current branch (`pull.rebase=false` → merge).
 - Then `git push --tags`.
 - **No `git push --force` / `--force-with-lease` anywhere** → §11.4.113-safe.
-

2. Live git evidence (cadence, identity, remotes)

`git log -15` (see `evidence/daemon/git_evidence.txt` for -20):

```
31cde9a1|Milos Vasic <i@mvasic.ru>|2026-07-05 13:35:03 +0500|Auto-commit
ada44c74|Milos Vasic <i@mvasic.ru>|2026-07-05 13:33:41 +0500|Auto-commit
4d25f404|Milos Vasic <i@mvasic.ru>|2026-07-05 13:31:02 +0500|Auto-commit
06f85ba7|Milos Vasic|2026-07-02 02:30:16 +0500|sync(submodules): bump heli
49ff34da|Milos Vasic|2026-07-02 02:15:26 +0500|feat(submodules): interrupt
...
1254e0a6|Test User <test@example.com>|2026-06-28 00:30:08 +0300|sync: auto
```

- **Total Auto-commit commits in history:** 181.
- **Cadence:** irregular. On 2026-07-05: 13:31 → 13:33 → 13:35 (~1.5–2.5 min apart, a short burst), then **nothing on 2026-07-06**, prior activity 2026-07-02. A fixed-interval OS daemon would produce a continuous, regular stream — this does not.
- **Author of the Auto-commit burst:** Milos Vasic <i@mvasic.ru> — identical to this repo’s configured identity (`git config user.email` → `i@mvasic.ru`), i.e. the interactive/agent identity on **this** host.
- The older Test User <test@example.com> “sync:” commits are a **different identity/host** (cross-host rsync sync path).

Remotes (`git remote -v`):

```
github    git@github.com:HelixDevelopment/Helix-CLI.git (fetch/push)
gitlab    git@gitlab.com:helixdevelopment1/HelixCode.git (fetch/push)
origin    git@github.com:HelixDevelopment/HelixCode.git (fetch)
origin    git@github.com:HelixDevelopment/Helix-CLI.git (push)
origin    git@gitlab.com:helixdevelopment1/HelixCode.git (push)
upstream  git@github.com:HelixDevelopment/Helix-CLI.git (fetch/push)
```

`git config --get-all remote.origin.url` (fetch) =

`git@github.com:HelixDevelopment/HelixCode.git`. origin has **two push URLs** (Helix-CLI GitHub + HelixCode GitLab) — a fan-out remote. All SSH (§Rule 3-compatible).

Active hooks: `.git/hooks/` contains **no** non-`.sample` files → no pre/post-commit hook interference.

3. Does it descend into / clobber submodules?

- The committer runs `git add . && git commit` **only in the CWD repo**. It does **NOT** iterate submodules (no `git submodule foreach`, no `cd` into submodules).
 - Consequence: if invoked from the **meta-repo root** it will `git add .` the **submodule GITLINK pointer bumps** that are already dirty in the worktree (e.g. current `git status` shows `M constitution, m submodules/helix_qa`) and commit them onto the current branch. It does not *create* pointer changes, but it will **sweep whatever is dirty** into the commit (a §11.4.84 quiescence hazard, not a branch hazard).
 - It does **not** reset or check out submodules.
-

4. What it does NOT do (proven by reading the whole chain)

- No `git checkout / git switch / git branch -f / git reset` — cannot move you off your branch or reset it.
 - No hard-coded `main / master` target — commits to `HEAD`.
 - No `--force push`.
 - No `git worktree / git clean -df` on the tree.
 - No commit *inside* submodules.
-

5. Is there actually a periodic DAEMON? — UNCONFIRMED: (no OS daemon found)

Exhaustive negative evidence (`evidence/daemon/no_os_daemon_evidence.txt`): `- crontab -l` → **no crontab for milos**. `- systemctl --user list-timers --all` → only `systemd-tmpfiles-clean.timer` + `gocryptfs-fsck.timer` (**no** `commit/git/sync` timer). `- System-level systemctl list-timers --all | grep -iE 'commit|helix|sync|git'` → **none**. `- atq` → **no files in queue**. `- pgrep -af 'watchexec|inotifywait|entr|fswatch'` → **none**. `- pgrep -af 'commit|watch|auto'` → only `tmx-recycler.sh` watch processes, which are the **tmux idle-session recycler** (read its header: purpose is killing idle detached tmux sessions; `grep 'commit|git' tmx-recycler.sh` → empty). **Not** a committer. `- No .bashrc`

alias/function/SESSION export tied to auto-commit; no script under ~/scripts, ~/tmux/scripts, ~/.config references the helix path AND git add/commit.

Conclusion (evidence-based): On THIS host there is **no fixed-interval OS-level auto-commit daemon**. The Auto-commit commits are produced by **on-demand invocations of the commit toolkit command by an external orchestrator**.

Running candidates (evidence: pgrep -af claude): three live Claude Code agent sessions — helix_code (pid 1166332), helix_terminator (pid 849723), atmosphere_t1 (pid 977834). These agent loops (or a cross-host rsync sync for the Test User commits) are the most plausible trigger.

UNCONFIRMED: the exact caller of commit for the 2026-07-05 burst. **What would confirm it:** (a) instrument the toolkit — temporarily wrap Software-Toolkit/Utils/Git/commit.sh to log \$PPID + ps -o command= -p \$PPID + pwd + date to a file, then observe the next Auto-commit; or (b) grep the running agent sessions' transcripts/logs for commit/cmt invocations; or (c) ask the operator which agent/session runs the auto-commit loop. Do **not** guess the trigger.

6. SAFE feature-branch-creation procedure for §11.4.181

Core safety fact: the committer commits to and pushes the **current branch**, with **no checkout/reset/branch-switch and no force-push**. Therefore, once HEAD is on feature/helixllm-full-extension, any Auto-commit fired by the orchestrator **lands on the feature branch**, not on main, and pushes the feature branch (not main). **No pause of the “daemon” is required** for branch-integrity — the mechanism is inherently branch-following.

Residual (non-branch) hazards to control, both from the broad git add .: 1. It will sweep **all currently-dirty files** (incl. the dirty constitution + submodules/helix_qa pointers, and the new docs/research/07.2026/ tree) into the first Auto-commit on the feature branch. 2. It pushes the feature branch to **all** fan-out upstreams automatically.

Recommended procedure (concrete, evidence-backed)

1. **Quiescence first (§11.4.84):** account for the current dirty tree BEFORE branching so the first auto git add . doesn't sweep unintended changes. git status currently shows M constitution, m submodules/helix_qa, ?? docs/research/07.2026/. Either commit/stash those deliberately on main (or under the feature branch) so their inclusion is intentional.
2. **Create + switch the branch yourself** (the toolkit never switches branches):

```
git switch -c feature/helixllm-full-extension
```

From this point HEAD is the feature branch; every `commit/git commit` (manual or orchestrator-fired) targets it. Verify with `git branch --show-current`.

3. **Work normally.** Any Auto-commit that fires now commits to `feature/helixllm-full-extension` and `push_all.sh` pushes THAT branch to the upstreams (creating it remotely on first push). `main` is never touched by the committer.
4. **If you want zero remote fan-out of the WIP feature branch** (optional, not required for safety): the toolkit's push is unconditional after commit, so to keep the feature branch local-only you would need to either (a) run work in a **git worktree** whose branch you never let the orchestrator push, or (b) temporarily point the orchestrator elsewhere. This is a *push-noise* preference, not a branch-safety requirement.
5. **Belt-and-braces isolation (strongest, optional) — git worktree:**

```
git worktree add ../helix_code-helixllm feature/helixllm-full-extension
```

If the auto-commit orchestrator only runs with CWD in the **main** checkout, a separate worktree fully decouples your feature work from any `git add .` fired against the main checkout (they are different working trees, same repo). Use this if §5's UNCONFIRMED: trigger makes you want hard isolation until the caller is identified.

Do we need to pause the daemon?

No — there is no OS daemon to pause, and the toolkit is branch-following (commits/pushes HEAD, never hard-targets main, never force-pushes). Branch integrity is preserved by simply being on the feature branch. Pause/worktree is only warranted if you want to avoid the broad `git add .` sweeping unrelated dirty files or auto-pushing the WIP branch — a hygiene choice, addressed by steps 1 + 5.

7. Evidence file index (./evidence/daemon/)

File	What it proves
cmt	repo-root wrapper (adds a period; not the auto source)

File	What it proves
Upstreamable__commit	PATH commit → delegates to commit.sh
Upstreamable__commit.sh	default MESSAGE="Auto-commit \$SESSION", empty SESSION → Auto-commit
SoftwareToolkit__Utils__Git__commit.sh	git add . + git commit -m (current branch) + push; no checkout/reset
SoftwareToolkit__Utils__Git__push_all.sh	pushes current branch to all upstream recipes + --tags; no --force
git_evidence.txt	log/cadence/identity/remotes/hooks snapshot
no_os_daemon_evidence.txt	crontab/systemd/at/file-watcher all negative